MAS1004 Data & AI, 2026 Fall

Assignment 1: Long report

Name: Garatti Giulia Student ID: 20241184

Problem 1. Categories and images

Why these categories

I chose these four categories because they are types of quartz, characterised by different colours. I'm interested in crystals, I own at least one crystal of each category, and they are the ones I can usually distinguish easily. They are also things I can photograph myself, which Problem 5 needs.

I checked the four against `imagenet_classes.txt`, the 1,000 categories ResNet18 already tells apart: none of the four is in it, the only entries that share a word with them being "stone wall" and "ruddy turnstone", and nothing in the list is a piece of polished quartz. So the last layer has to be replaced and these four categories have to be learned from features that were trained on something else.

What I expected the model to use

Colour, above all, written down before training: purple is very different from green, so I thought amethyst and aventurine would not get mixed up, and rose quartz can be lighter in color, which makes it more similar to the light green of aventurine, so I expected rose quartz and aventurine to be the pair that does get mixed up. Shape I expected to be little help, because the pictures are polished pieces, all of them roundish pebbles with no points to go by.

Images per category

Asked `collect.py` for 150 images in each of four categories. Before any cleaning, in `data/raw`:

class	asked for	came back
amethyst	150	134
aventurine	150	138
citrine	150	150
rose_quartz	150	150
total	600	572

Amethyst and aventurine came back short because `collect.py` skips a link that does not open, is not an image, or is too small to use, and counts what it skipped. The 134, 138, 150, 150 are the `downloaded` column of `clean.py count` (`outputs/clean_count.txt`), the counts as they were before anything was removed; the 21 images for Problem 5, five in each class and six rose quartz, are printed by `check.py` in the Appendix. Every image is stored as `data/raw/<class name>/0001.jpg` onwards, and `data/clean` was a copy of `data/raw` until `clean.py remove` started moving files out of it.

Problem 2. Preparing an image



as downloaded (`rose_quartz/0060`): a rose quartz heart on a stand, with a sphere and two towers behind it



what prepare_image made of it: the middle 224 × 224 square

`prepare_image` prepares every photo the way ResNet18 expects: it shrinks the photo so that its shorter side is 256 pixels, keeps only the middle 224 × 224 square, and scales the colours. Everything outside that square is thrown away.

An image where the crop cut off part of the thing I care about is the one above, `rose_quartz/0060`. The red square below is what the model keeps: the bottom of the heart and the top of the tall tower are cut off.



How much is thrown away depends on the shape of the photo. For one of my own photos, `aventurine/IMG_3068.JPG` (4032 × 3024), the numbers are:

step	result
on disk	4032 × 3024
after shrinking the shorter side to 256	341 × 256
square kept	x 58..282, y 16..240
cut off, in the 341 × 256 picture	58 left, 59 right, 16 top, 16 bottom
kept	57.5% of the area

So 42.5% of that photo never reaches the model, mostly from the left and right sides; there the stone is in the middle, so it is not cut.

The same holds for my other photos, where I placed the stone in the middle. Below are two of them, `rose_quartz/9dacd40a` (left) and `rose_quartz/ea7f4b76` (right), with the square that `prepare_image` keeps drawn in red: in both, the whole stone is inside the square, unlike in the downloaded picture above.



Four of my 21 photos are stored sideways: `amethyst/IMG_3200`, `aventurine/IMG_3196`, `rose_quartz/IMG_3069` and `rose_quartz/IMG_3194`. The phone saved them lying on their side, with a note in the file that says "turn this photo before showing it". `prepare_image` does not read that note, so in Python the model sees these four photos turned on their side, while the web page does read it and shows them upright. For these four photos, Python and the page therefore do not see quite the same picture.

Problem 3. First training runs

tag	start	trained	epochs	lr	trainable parameters	train acc.	test acc.
run	ImageNet	all layers	10	0.0001	11,178,564	100.0%	92.2%
scratch	random	all layers	10	0.0001	11,178,564	99.6%	82.6%
frozen	ImageNet	last layer	10	0.001	2,052	97.2%	88.7%
lr1e-3	ImageNet	all layers	10	0.001	11,178,564	96.7%	84.3%
clean	ImageNet	all layers	10	0.0001	11,178,564	100.0%	100.0%

The first four rows were trained before any cleaning, on the same 572 images split the same way: 457 for training and 115 for testing. So their test accuracies can be compared with each other. The fifth row, `clean`, uses the same settings as `run`, but after the cleaning of Problem 4: 374 images, 299 for training and 75 for testing. It is compared with `run` in Problems 4 and 5.

With 115 test images, one image is 0.87%. Training twice with the same settings does not give exactly the same result, because the order of the images and the starting values of the new last layer are random. I ran the first three commands twice (the first attempt's output is below; the table uses the second), and the test accuracy changed by 2.6, 1.7 and 3.5 points, that is 3, 2 and 4 test images. So a difference of up to about four test images between two runs does not mean much.

```
reading /content/drive/MyDrive/mas1004/assignment1/data/clean
572 images, 4 classes, read in 17 s
  amethyst                 134
  aventurine               138
  citrine                  150
  rose_quartz              150
training on 457, testing on 115
model has 11,178,564 parameters, 11,178,564 of them will be trained
/content/drive/MyDrive/mas1004/assignment1/src/train.py:127: UserWarning: Converting a tensor with
↪  requires_grad=True to a scalar may lead to unexpected behavior.
```

```
Consider using tensor.detach() first. (Triggered internally at
↪  /pytorch/torch/csrc/autograd/generated/python_variable_methods.cpp:836.)
  losses.append(float(loss))
epoch   1/10  train loss 0.8230  train acc  65.0%  test loss 0.5497  test acc  84.3%
epoch   2/10  train loss 0.1493  train acc  97.6%  test loss 0.3013  test acc  91.3%
epoch   3/10  train loss 0.0564  train acc  99.8%  test loss 0.3036  test acc  88.7%
epoch   4/10  train loss 0.0268  train acc 100.0%  test loss 0.2705  test acc  87.0%
epoch   5/10  train loss 0.0141  train acc 100.0%  test loss 0.2626  test acc  89.6%
epoch   6/10  train loss 0.0551  train acc  99.8%  test loss 0.2851  test acc  90.4%
epoch   7/10  train loss 0.0199  train acc 100.0%  test loss 0.2632  test acc  91.3%
epoch   8/10  train loss 0.0127  train acc 100.0%  test loss 0.2936  test acc  89.6%
epoch   9/10  train loss 0.0074  train acc 100.0%  test loss 0.2620  test acc  91.3%
epoch  10/10  train loss 0.0089  train acc 100.0%  test loss 0.2572  test acc  89.6%

trained in 0.3 minutes
train accuracy 100.0%
test  accuracy 89.6%
wrote /content/drive/MyDrive/mas1004/assignment1/results/run_curves.png
wrote /content/drive/MyDrive/mas1004/assignment1/results/run_confusion.png
wrote /content/drive/MyDrive/mas1004/assignment1/results/run_worst.png
wrote /content/drive/MyDrive/mas1004/assignment1/results/run_model.pt

One line for your experiment table:
| run | ImageNet | all layers | 10 | 0.0001 | 11,178,564 | 100.0% | 89.6% |

To put this model on your web page: python src/export_web.py --tag run
reading /content/drive/MyDrive/mas1004/assignment1/data/clean
572 images, 4 classes, read in 17 s
  amethyst                134
  aventurine              138
  citrine                 150
  rose_quartz             150
training on 457, testing on 115
model has 11,178,564 parameters, 11,178,564 of them will be trained
/content/drive/MyDrive/mas1004/assignment1/src/train.py:127: UserWarning: Converting a tensor with
↪  requires_grad=True to a scalar may lead to unexpected behavior.
Consider using tensor.detach() first. (Triggered internally at
↪  /pytorch/torch/csrc/autograd/generated/python_variable_methods.cpp:836.)
  losses.append(float(loss))
epoch   1/10  train loss 0.8317  train acc  68.1%  test loss 0.8228  test acc  71.3%
epoch   2/10  train loss 0.6004  train acc  75.9%  test loss 0.6312  test acc  72.2%
epoch   3/10  train loss 0.4126  train acc  85.8%  test loss 0.5793  test acc  77.4%
epoch   4/10  train loss 0.3719  train acc  88.4%  test loss 0.4506  test acc  80.9%
epoch   5/10  train loss 0.2555  train acc  90.8%  test loss 0.6168  test acc  78.3%
epoch   6/10  train loss 0.2748  train acc  94.3%  test loss 0.8744  test acc  71.3%
epoch   7/10  train loss 0.2409  train acc  90.8%  test loss 0.7762  test acc  81.7%
epoch   8/10  train loss 0.1704  train acc  95.8%  test loss 0.6489  test acc  83.5%
epoch   9/10  train loss 0.0924  train acc  97.6%  test loss 0.5353  test acc  84.3%
epoch  10/10  train loss 0.0610  train acc  98.7%  test loss 0.6912  test acc  80.9%

trained in 0.3 minutes
train accuracy 100.0%
test  accuracy 80.9%
wrote /content/drive/MyDrive/mas1004/assignment1/results/scratch_curves.png
wrote /content/drive/MyDrive/mas1004/assignment1/results/scratch_confusion.png
wrote /content/drive/MyDrive/mas1004/assignment1/results/scratch_worst.png
wrote /content/drive/MyDrive/mas1004/assignment1/results/scratch_model.pt

One line for your experiment table:
| scratch | random | all layers | 10 | 0.0001 | 11,178,564 | 100.0% | 80.9% |

To put this model on your web page: python src/export_web.py --tag scratch
reading /content/drive/MyDrive/mas1004/assignment1/data/clean
572 images, 4 classes, read in 17 s
  amethyst                134
  aventurine              138
  citrine                 150
```

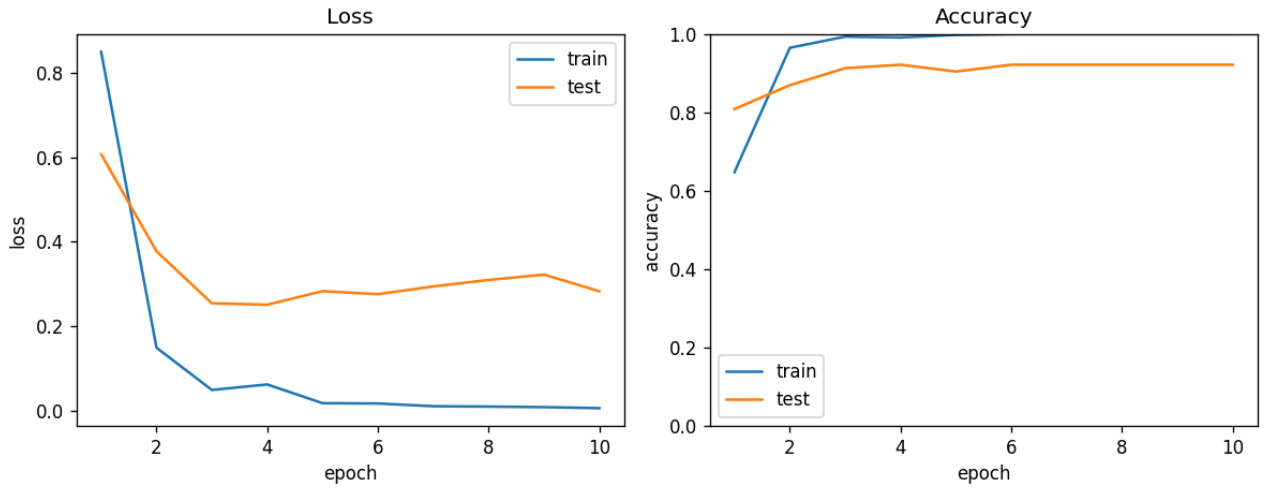
```
  rose_quartz              150
training on 457, testing on 115
model has 11,178,564 parameters, 2,052 of them will be trained
/content/drive/MyDrive/mas1004/assignment1/src/train.py:127: UserWarning: Converting a tensor with
↪  requires_grad=True to a scalar may lead to unexpected behavior.
Consider using tensor.detach() first. (Triggered internally at
↪  /pytorch/torch/csrc/autograd/generated/python_variable_methods.cpp:836.)
  losses.append(float(loss))
epoch   1/10  train loss 1.2975  train acc  38.1%  test loss 1.0235  test acc  67.8%
epoch   2/10  train loss 0.9246  train acc  70.9%  test loss 0.7974  test acc  79.1%
epoch   3/10  train loss 0.7215  train acc  80.1%  test loss 0.6948  test acc  81.7%
epoch   4/10  train loss 0.5778  train acc  85.3%  test loss 0.5952  test acc  84.3%
epoch   5/10  train loss 0.5307  train acc  86.9%  test loss 0.5509  test acc  86.1%
epoch   6/10  train loss 0.4672  train acc  89.5%  test loss 0.5060  test acc  86.1%
epoch   7/10  train loss 0.4013  train acc  91.5%  test loss 0.4750  test acc  87.0%
epoch   8/10  train loss 0.3658  train acc  91.5%  test loss 0.4738  test acc  87.8%
epoch   9/10  train loss 0.3396  train acc  92.6%  test loss 0.4316  test acc  86.1%
epoch  10/10  train loss 0.3364  train acc  93.7%  test loss 0.4312  test acc  85.2%

trained in 0.1 minutes
train accuracy 96.3%
test  accuracy 85.2%
wrote /content/drive/MyDrive/mas1004/assignment1/results/frozen_curves.png
wrote /content/drive/MyDrive/mas1004/assignment1/results/frozen_confusion.png
wrote /content/drive/MyDrive/mas1004/assignment1/results/frozen_worst.png
wrote /content/drive/MyDrive/mas1004/assignment1/results/frozen_model.pt

One line for your experiment table:
| frozen | ImageNet | last layer | 10 | 0.001 | 2,052 | 96.3% | 85.2% |

To put this model on your web page: python src/export_web.py
```

The loss curve

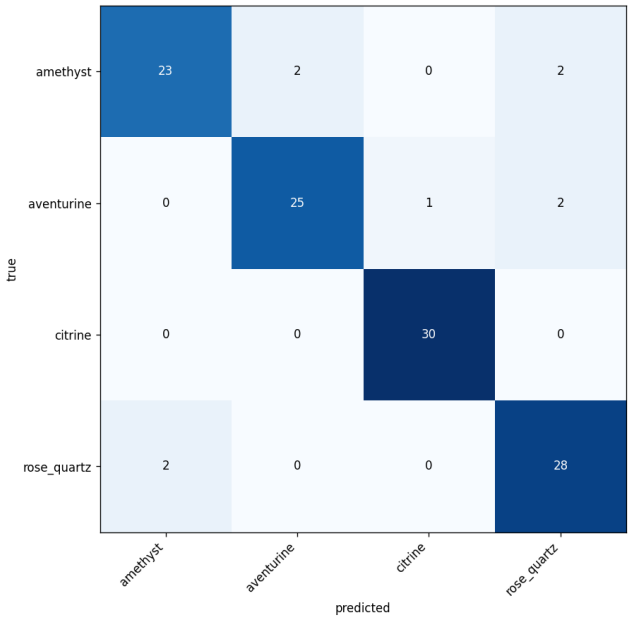


In the `run` curve the training loss goes down from 0.8494 at epoch 1 to 0.0054 at epoch 10, with one small rise at epoch 4 (0.0485 to 0.0615). After epoch 5 it is below 0.02 and goes down only slowly, so it flattens out. The test loss is lowest at epoch 4 (0.2505) and then goes up a little, ending at 0.2822. The test accuracy reaches 92.2% at epoch 4 and does not improve after that.

The gap between train and test accuracy

The train accuracy is 100.0% and the test accuracy 92.2%, a gap of 7.8 points: all 457 training images are right, while 9 of the 115 test images are wrong. The model has learned its training images by heart; the test accuracy is the number that says how well it does on pictures it has not seen.

The confusion matrix



Rows are what the image really is, columns what the model said. Citrine is always right (30 of 30). The 9 mistakes are: amethyst called aventurine 2 times and rose quartz 2 times, aventurine called rose quartz 2 times and citrine once, and rose quartz called amethyst 2 times. Counting both directions, the pairs mixed up most are amethyst with rose quartz (4) and aventurine with rose quartz (4).

My answer to "which two categories, and did it surprise you":

> The categories that got mixed up and surprised me were amethyst turning out as aventurine twice. That is because purple is very different from green, so I thought they wouldn't get mixed. Aventurine was mixed up for rose quartz also twice but that surprised me less compared to the amethyst turning out as aventurine, because rose quartz can be lighter in color, which makes it more similar to the light green of aventurine. I wasn't surprised about amethyst and rose quartz to get mixed up due to their similar colors.

Starting from ImageNet against starting from random numbers

With the same images, code and number of epochs, starting from ImageNet gives 92.2% and starting from random numbers 82.6%: 9.6 points, 11 test images apart, more than the noise of about four images. Why I think that is: considering my categories, and that I expected the model to tell them apart by colour, starting from random numbers needs much longer to learn than starting from ImageNet, which is already trained on millions of images. In the same 10 epochs the random model only has my 457 training images to learn from, and its test loss goes up and down (1.2557 at epoch 7) while its train accuracy keeps rising.

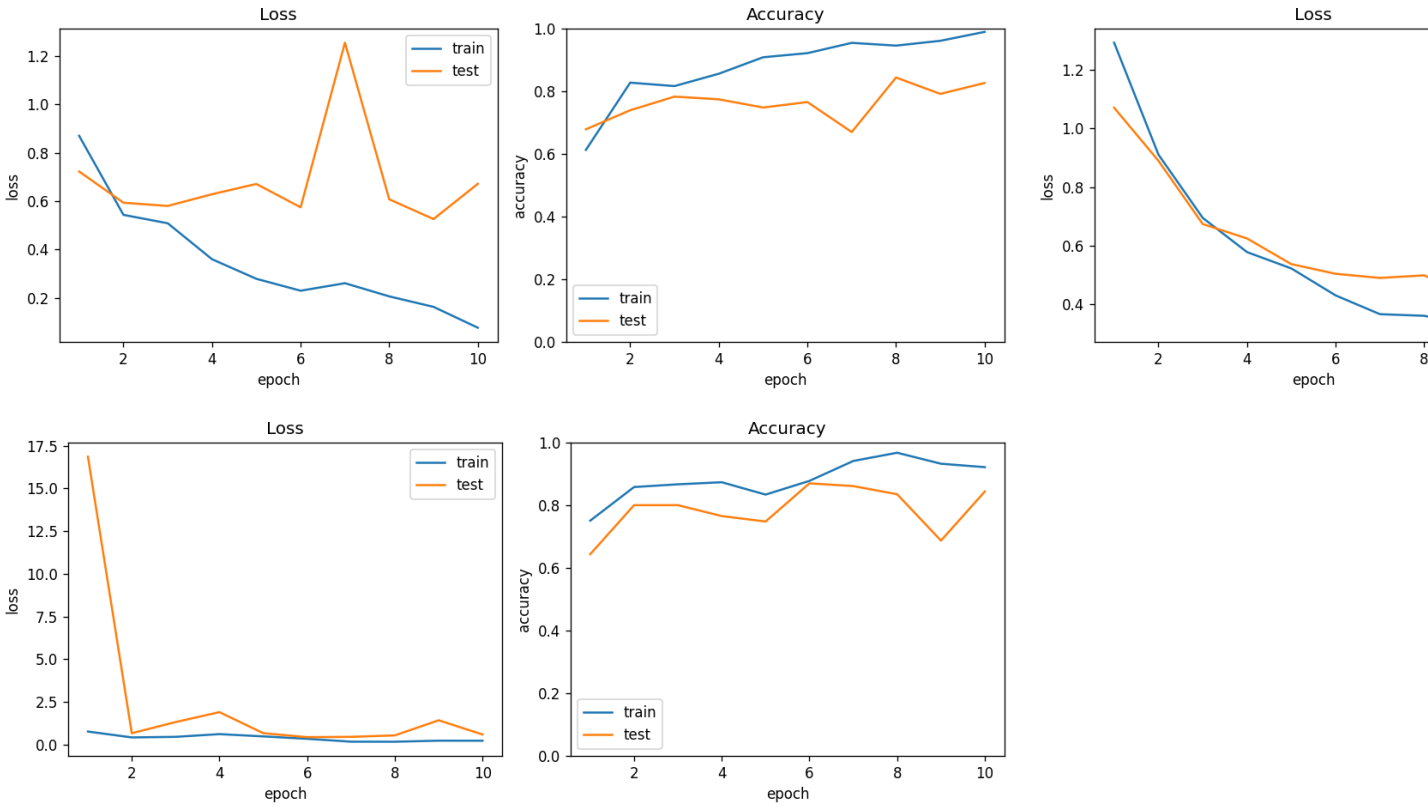
Training only the last layer

Training only the new last layer changes 2,052 of the 11,178,564 parameters, about 0.02%. It reaches 88.7%, against 92.2% when all layers are trained: 3.5 points, 4 test images, which is within the noise. It also trains faster (0.1 minutes instead of 0.3).

My own setting: a larger learning rate

`lr1e-3` is the same as `run` with a learning rate ten times larger (0.001 instead of 0.0001). It reaches only 84.3%, and its test loss jumps around (16.8645 at epoch 1, 1.8993 at epoch 4, 1.4221 at epoch 9). A learning rate that works for training one new layer is too large when all the layers are trained.

The curves of the other three runs, `scratch`, `frozen` and `lr1e-3`, top to bottom:



Output of the runs

```
reading /content/drive/MyDrive/mas1004/assignment1/data/clean
572 images, 4 classes, read in 17 s
  amethyst                134
  aventurine              138
  citrine                 150
  rose_quartz             150
training on 457, testing on 115
model has 11,178,564 parameters, 11,178,564 of them will be trained
epoch   1/10  train loss 0.8494  train acc  64.8%  test loss 0.6067  test acc  80.9%
epoch   2/10  train loss 0.1486  train acc  96.5%  test loss 0.3769  test acc  87.0%
epoch   3/10  train loss 0.0485  train acc  99.3%  test loss 0.2537  test acc  91.3%
epoch   4/10  train loss 0.0615  train acc  99.1%  test loss 0.2505  test acc  92.2%
epoch   5/10  train loss 0.0172  train acc  99.8%  test loss 0.2826  test acc  90.4%
epoch   6/10  train loss 0.0164  train acc 100.0%  test loss 0.2754  test acc  92.2%
epoch   7/10  train loss 0.0100  train acc 100.0%  test loss 0.2938  test acc  92.2%
epoch   8/10  train loss 0.0091  train acc 100.0%  test loss 0.3091  test acc  92.2%
epoch   9/10  train loss 0.0078  train acc 100.0%  test loss 0.3218  test acc  92.2%
epoch  10/10  train loss 0.0054  train acc 100.0%  test loss 0.2822  test acc  92.2%

trained in 0.3 minutes
train accuracy 100.0%
test  accuracy 92.2%
wrote /content/drive/MyDrive/mas1004/assignment1/results/run_curves.png
wrote /content/drive/MyDrive/mas1004/assignment1/results/run_confusion.png
wrote /content/drive/MyDrive/mas1004/assignment1/results/run_worst.png
wrote /content/drive/MyDrive/mas1004/assignment1/results/run_model.pt

One line for your experiment table:
| run | ImageNet | all layers | 10 | 0.0001 | 11,178,564 | 100.0% | 92.2% |

To put this model on your web page: python src/export_web.py --tag run
```

```
reading /content/drive/MyDrive/mas1004/assignment1/data/clean
572 images, 4 classes, read in 19 s
  amethyst                134
  aventurine              138
  citrine                 150
  rose_quartz             150
training on 457, testing on 115
model has 11,178,564 parameters, 11,178,564 of them will be trained
epoch   1/10  train loss 0.8704  train acc  61.3%  test loss 0.7225  test acc  67.8%
epoch   2/10  train loss 0.5431  train acc  82.7%  test loss 0.5934  test acc  73.9%
epoch   3/10  train loss 0.5083  train acc  81.6%  test loss 0.5800  test acc  78.3%
epoch   4/10  train loss 0.3594  train acc  85.6%  test loss 0.6284  test acc  77.4%
epoch   5/10  train loss 0.2784  train acc  90.8%  test loss 0.6710  test acc  74.8%
epoch   6/10  train loss 0.2291  train acc  92.1%  test loss 0.5744  test acc  76.5%
epoch   7/10  train loss 0.2603  train acc  95.4%  test loss 1.2557  test acc  67.0%
epoch   8/10  train loss 0.2057  train acc  94.5%  test loss 0.6075  test acc  84.3%
epoch   9/10  train loss 0.1623  train acc  96.1%  test loss 0.5257  test acc  79.1%
epoch  10/10  train loss 0.0759  train acc  98.9%  test loss 0.6717  test acc  82.6%

trained in 0.3 minutes
train accuracy 99.6%
test  accuracy 82.6%
wrote /content/drive/MyDrive/mas1004/assignment1/results/scratch_curves.png
wrote /content/drive/MyDrive/mas1004/assignment1/results/scratch_confusion.png
wrote /content/drive/MyDrive/mas1004/assignment1/results/scratch_worst.png
wrote /content/drive/MyDrive/mas1004/assignment1/results/scratch_model.pt

One line for your experiment table:
| scratch | random | all layers | 10 | 0.0001 | 11,178,564 | 99.6% | 82.6% |

To put this model on your web page: python src/export_web.py --tag scratch
```

```
reading /content/drive/MyDrive/mas1004/assignment1/data/clean
572 images, 4 classes, read in 19 s
  amethyst                134
  aventurine              138
  citrine                 150
  rose_quartz             150
training on 457, testing on 115
model has 11,178,564 parameters, 2,052 of them will be trained
epoch   1/10  train loss 1.2926  train acc  40.0%  test loss 1.0710  test acc  53.9%
epoch   2/10  train loss 0.9113  train acc  71.8%  test loss 0.8902  test acc  68.7%
epoch   3/10  train loss 0.6955  train acc  81.6%  test loss 0.6744  test acc  84.3%
epoch   4/10  train loss 0.5789  train acc  86.4%  test loss 0.6253  test acc  82.6%
epoch   5/10  train loss 0.5232  train acc  85.8%  test loss 0.5379  test acc  87.0%
epoch   6/10  train loss 0.4312  train acc  91.7%  test loss 0.5049  test acc  87.8%
epoch   7/10  train loss 0.3673  train acc  92.6%  test loss 0.4909  test acc  89.6%
epoch   8/10  train loss 0.3617  train acc  91.0%  test loss 0.4995  test acc  87.0%
epoch   9/10  train loss 0.3320  train acc  90.6%  test loss 0.4328  test acc  90.4%
epoch  10/10  train loss 0.3211  train acc  92.6%  test loss 0.4390  test acc  88.7%

trained in 0.1 minutes
train accuracy 97.2%
test  accuracy 88.7%
wrote /content/drive/MyDrive/mas1004/assignment1/results/frozen_curves.png
wrote /content/drive/MyDrive/mas1004/assignment1/results/frozen_confusion.png
wrote /content/drive/MyDrive/mas1004/assignment1/results/frozen_worst.png
wrote /content/drive/MyDrive/mas1004/assignment1/results/frozen_model.pt

One line for your experiment table:
| frozen | ImageNet | last layer | 10 | 0.001 | 2,052 | 97.2% | 88.7% |

To put this model on your web page: python src/export_web.py --tag frozen
```

```
reading /content/drive/MyDrive/mas1004/assignment1/data/clean
572 images, 4 classes, read in 19 s
  amethyst                 134
  aventurine               138
  citrine                  150
  rose_quartz              150
training on 457, testing on 115
model has 11,178,564 parameters, 11,178,564 of them will be trained
epoch   1/10  train loss 0.7630  train acc  75.1%  test loss 16.8645  test acc  64.3%
epoch   2/10  train loss 0.4186  train acc  85.8%  test loss 0.6641  test acc  80.0%
epoch   3/10  train loss 0.4512  train acc  86.7%  test loss 1.3179  test acc  80.0%
epoch   4/10  train loss 0.6116  train acc  87.3%  test loss 1.8993  test acc  76.5%
epoch   5/10  train loss 0.4776  train acc  83.4%  test loss 0.6656  test acc  74.8%
epoch   6/10  train loss 0.3374  train acc  87.7%  test loss 0.4330  test acc  87.0%
epoch   7/10  train loss 0.1688  train acc  94.1%  test loss 0.4516  test acc  86.1%
epoch   8/10  train loss 0.1660  train acc  96.7%  test loss 0.5370  test acc  83.5%
epoch   9/10  train loss 0.2274  train acc  93.2%  test loss 1.4221  test acc  68.7%
epoch  10/10  train loss 0.2240  train acc  92.1%  test loss 0.5896  test acc  84.3%

trained in 0.3 minutes
train accuracy 96.7%
test  accuracy 84.3%
wrote /content/drive/MyDrive/mas1004/assignment1/results/lr1e-3_curves.png
wrote /content/drive/MyDrive/mas1004/assignment1/results/lr1e-3_confusion.png
wrote /content/drive/MyDrive/mas1004/assignment1/results/lr1e-3_worst.png
wrote /content/drive/MyDrive/mas1004/assignment1/results/lr1e-3_model.pt

One line for your experiment table:
| lr1e-3 | ImageNet | all layers | 10 | 0.001 | 11,178,564 | 96.7% | 84.3% |

To put this model on your web page: python src/export_web.py --tag lr1e-3
```

Problem 4. Cleaning

My rule

Written down before any `clean.py remove` ran, while `data/removed/` was still empty, and not tidied afterwards:

> My rules:
>
> I remove images that don't show a crystal at all
>
> I remove images that show drawings
>
> I remove images that show jewellery where the crystal is not predominant
>
> I remove images that has text and that take much space in the photo.
>
> I remove images that don't show clearly the crystal or not in the right direction.
>
> I remove images that has more than one type of crystal
>
> I remove images that show the wrong crystal.

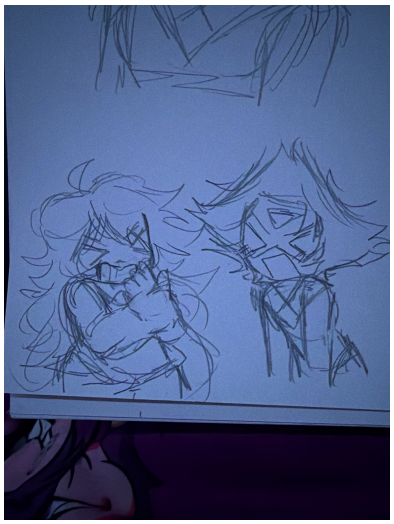
How I cleaned, in order: I wrote these rules first. Then I went through all 572 downloaded images myself and listed every image that broke a rule, next to the rule it broke. Then I looked at the suspect sheets and the near copies. In the end I removed only the images on my list (188) and the near copies (10), each with its reason, using `clean.py remove`.

Three kinds of junk

Mostly unrelated objects: clothes, car accessories, logos, drawings, and also characters and the wrong crystal. Three kinds, each with one of the removed files:

- not a crystal at all: `aventurine/0102.jpg`, a logo (left)
- a drawing rather than a photograph: `amethyst/0118.jpg` (middle)
- the wrong crystal: `aventurine/0110.jpg`, red stones (right)







What I removed

```
class                    downloaded  removed   kept  other
amethyst                        134       40     94      0
aventurine                      138       83     55      0
citrine                         150       17    133      0
rose_quartz                     150       58     92      0

reasons
    86  The image doesn't show a crystal at all
    25  The image shows the wrong crystal
    25  The image has text and that take much space of the photo
    22  The image shows jewelry where the crystal is not predominant
    18  The image has more than one type of crystal
    10  The image doesn't show clearly the crystal, or not in the right direction
    10  The image is a near copy of another picture
     2  The image is a drawing

"other" is images that are in data/raw but neither in data/clean nor removed with `clean.py
↪  remove`, for example ones you deleted by hand. Their reasons are not recorded.
```

Every removed file, with its reason, is listed in `outputs/removed_log.csv`; the per-class numbers in this section are counted from it.

198 of the 572 images were removed and 374 kept. Aventurine changed most (83 of 138 removed), mostly images that did not show a crystal at all (45) or had a lot of text (16), and citrine least (17 of 150).

Near copies

The tool found 27 pairs of near copies. In 16 of them both pictures had already been removed because they broke one of my rules. For the other 9 groups I kept one picture and removed the other 10, with the reason “The image is a near copy of another picture”:

kept	the near copies removed
`citrine/0100.jpg`	`citrine/0018.jpg`
`citrine/0149.jpg`	`citrine/0032.jpg`
`citrine/0006.jpg`	`citrine/0069.jpg`
`citrine/0050.jpg`	`citrine/0099.jpg`
`citrine/0083.jpg`	`citrine/0041.jpg`
`rose_quartz/0144.jpg`	`rose_quartz/0150.jpg`
`amethyst/0033.jpg`	`amethyst/0055.jpg`
`amethyst/0022.jpg`	`amethyst/0031.jpg`
`aventurine/0134.jpg`	`aventurine/0123.jpg`, `aventurine/0132.jpg`

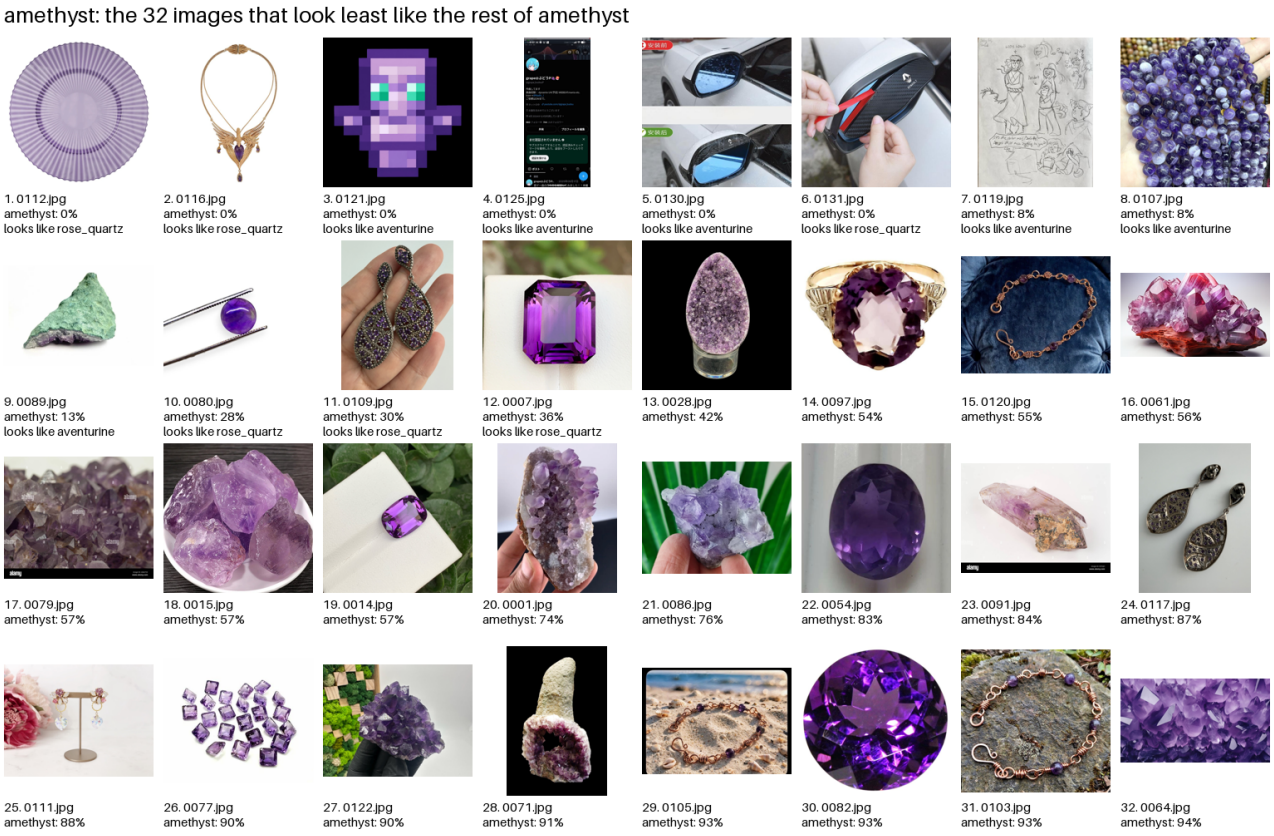
The suspects

Each suspect sheet shows the 32 images of a class that look least like the rest of that class (`outputs/suspects.csv` gives every image its rank, rank 1 being the least typical). This table shows, for the images I removed by my rules, where they were on that ranking:

class	removed by my rules	rank 1–20	rank 21–32	rank 33 and later
amethyst	38	11	6	21
aventurine	81	15	9	57
citrine	12	4	0	8
rose_quartz	57	15	6	36

Of the first 20 suspects in each class, I removed 11 amethyst, 15 aventurine, 4 citrine and 15 rose quartz.

Did the suspects include junk I had missed when I looked myself? No. I had already listed every image that broke my rules, and I removed only those and the near copies. The suspect sheets also showed images that the tool found unusual but that I had judged ok when I first went through them, because they did not break my rules, so I kept them. For example, `amethyst/0107.jpg` was on the amethyst suspect sheet, which gave it 8% amethyst, but it did not break any of my rules, so it is still in `data/clean`.



The amethyst suspect sheet, as `clean.py suspects` drew it. `amethyst/0107.jpg` is number 8, a strand of amethyst beads.

Did I find junk that the suspect sheets did not list? Yes, a lot: the last column of the table counts the images I removed that were not on the sheets at all (rank 33 and later), for example 57 of the 81 aventurine images I removed.

Before and after cleaning

```
MAS1004 Assignment 1, checking your work

Training images  (/content/drive/MyDrive/mas1004/assignment1/data/clean)
------------------------------------------------------------------------
  amethyst                        94 images
  aventurine                      55 images
  citrine                        133 images
  rose_quartz                     92 images

Your images from a new source  (/content/drive/MyDrive/mas1004/assignment1/data/new_images)
-------------------------------------------------------------------------------------------
  amethyst                         5 images
  aventurine                       5 images
  citrine                          5 images
  rose_quartz                      6 images

The same image appearing twice
------------------------------
  ok       no file appears twice byte for byte. Near copies (the same picture resized or re-saved)
  ↪  are only found by `python src/clean.py suspects` and `python src/clean.py overlap`.

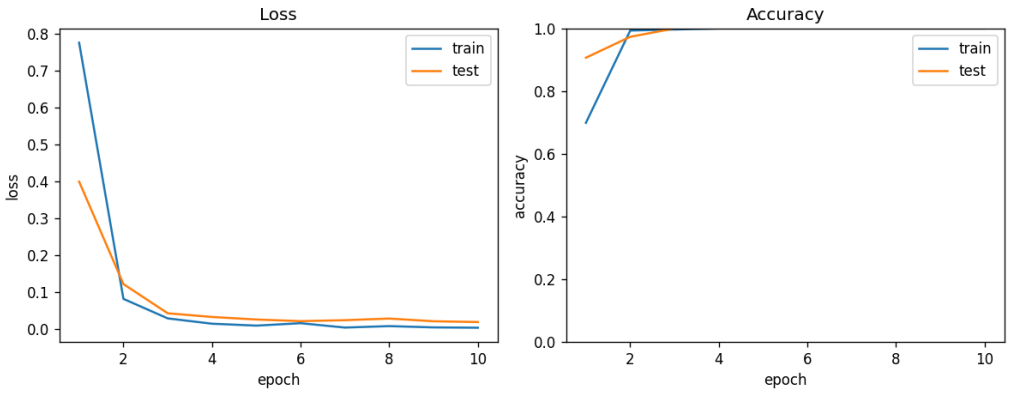
The model you exported for the web page
---------------------------------------
  PROBLEM  docs/ is missing model.onnx, model.json, selftest.json. Run src/run.py and then
  ↪  src/export_web.py first.

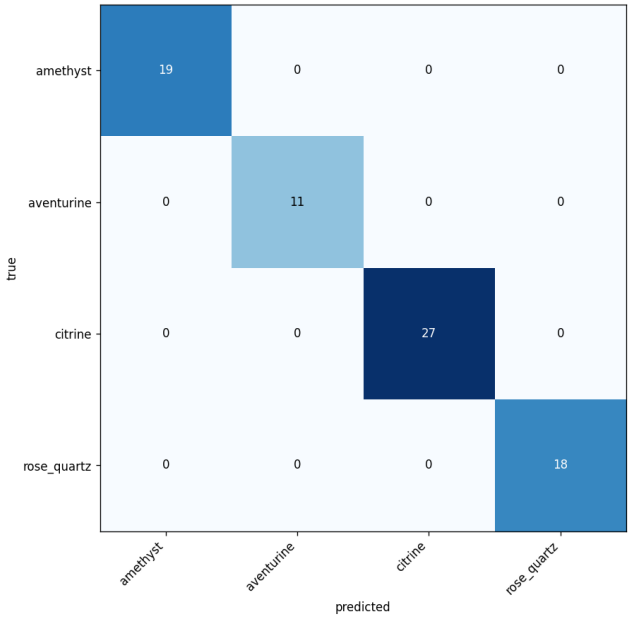
Your saved runs on your new images  (/content/drive/MyDrive/mas1004/assignment1/data/new_images)
-------------------------------------------------------------------------------------------------
  run               test accuracy   new images
  run                       92.2%        85.7%   (18/21)
  clean                    100.0%        85.7%   (18/21)

Summary
-------
  1 thing(s) to fix:
    - docs/ is missing model.onnx, model.json, selftest.json. Run src/run.py and then
    ↪  src/export_web.py first.
```

(The line about `docs/` missing the model was printed because the model had not been exported yet at that point; it was exported afterwards, see the Appendix.)

The `clean` run, its curves and its confusion matrix on its 75 test images:






```
reading /content/drive/MyDrive/mas1004/assignment1/data/clean
374 images, 4 classes, read in 136 s
  amethyst                94
  aventurine              55
  citrine                 133
  rose_quartz             92
training on 299, testing on 75
Downloading: "https://download.pytorch.org/models/resnet18-f37072fd.pth" to
↪  /root/.cache/torch/hub/checkpoints/resnet18-f37072fd.pth
model has 11,178,564 parameters, 11,178,564 of them will be trained
epoch   1/10  train loss 0.7755  train acc  69.9%  test loss 0.3989  test acc  90.7%
epoch   2/10  train loss 0.0811  train acc  99.3%  test loss 0.1212  test acc  97.3%
epoch   3/10  train loss 0.0282  train acc  99.7%  test loss 0.0421  test acc 100.0%
epoch   4/10  train loss 0.0138  train acc 100.0%  test loss 0.0321  test acc 100.0%
epoch   5/10  train loss 0.0086  train acc 100.0%  test loss 0.0252  test acc 100.0%
epoch   6/10  train loss 0.0152  train acc 100.0%  test loss 0.0208  test acc 100.0%
epoch   7/10  train loss 0.0034  train acc 100.0%  test loss 0.0234  test acc 100.0%
epoch   8/10  train loss 0.0073  train acc 100.0%  test loss 0.0278  test acc 100.0%
epoch   9/10  train loss 0.0038  train acc 100.0%  test loss 0.0205  test acc 100.0%
epoch  10/10  train loss 0.0029  train acc 100.0%  test loss 0.0183  test acc 100.0%

trained in 0.2 minutes
train accuracy 100.0%
test  accuracy 100.0%
wrote /content/drive/MyDrive/mas1004/assignment1/results/clean_curves.png
wrote /content/drive/MyDrive/mas1004/assignment1/results/clean_confusion.png
no mistakes to plot, which is suspicious. Check your split.
wrote /content/drive/MyDrive/mas1004/assignment1/results/clean_model.pt
```

```
One line for your experiment table:
| clean | ImageNet | all layers | 10 | 0.0001 | 11,178,564 | 100.0% | 100.0% |

To put this model on your web page: python src/export_web.py --tag clean
```

The test accuracy went from 92.2% (9 wrong of 115) to 100.0% (0 wrong of 75). Nothing went down, but this gain is smaller than it looks. The test set after cleaning is smaller (75 images instead of 115) and easier, because the unusual pictures that are hardest to classify were mostly removed by my rules. So part of the gain may come from a better model and part from an easier test, and these two numbers cannot tell them apart. (With no mistakes on the test set, `run.py` printed "no mistakes to plot, which is suspicious. Check your split." and drew no picture of mistakes for this run.)

The fair comparison is on my 21 new photos, which did not change: 18 of 21 right before cleaning and 18 of 21 after. Cleaning did not change the number of right answers on them, but, as Problem 5 shows, it changed which photos were right.

Problem 5. Images from a new source

My new images are 21 photographs of crystals, five of each class and six of rose quartz, taken for this assignment by me and by family members and friends I asked for them. They are saved as JPEG, because the code cannot read the phones' HEIC files, in `data/new_images/<class>/`.

I am sure none of them is among my downloads, because they were never online: they were taken for this purpose, and the downloads come from a web search. The overlap check confirms it: it compared each of my 21 photos with all the downloaded images, and none of them is a near copy (the closest any of them comes is 0.85, and near copies are above 0.95).

```
none of your 21 new images is a near copy of any of the 570 images you downloaded

the closest any new image comes to a downloaded one: 0.85 (near copies are above 0.95)
```

The accuracy on the downloaded test set and on my new images, side by side:

run	downloaded test set	my new images
`run` (before cleaning)	92.2%	85.7% (18/21)
`clean` (after cleaning)	100.0%	85.7% (18/21)

The two test accuracies are measured on different test sets (115 images before cleaning, 75 after; see Problem 4), while the new images are the same 21 photos for both. On my new images both models get 18 of 21 right, less than on the downloaded test set.

The mistakes it was confident about

`check.py` lists the five mistakes the published model was most sure about; on my new images it made only three, so these are all of them. All three are citrines:



left to right: `6dc1dd3c` said rose quartz 54%, `58f05f6d` said rose quartz 51%, `2788e4fa` said amethyst 32%; all three really citrine.

What I think pushed each of them the wrong way:

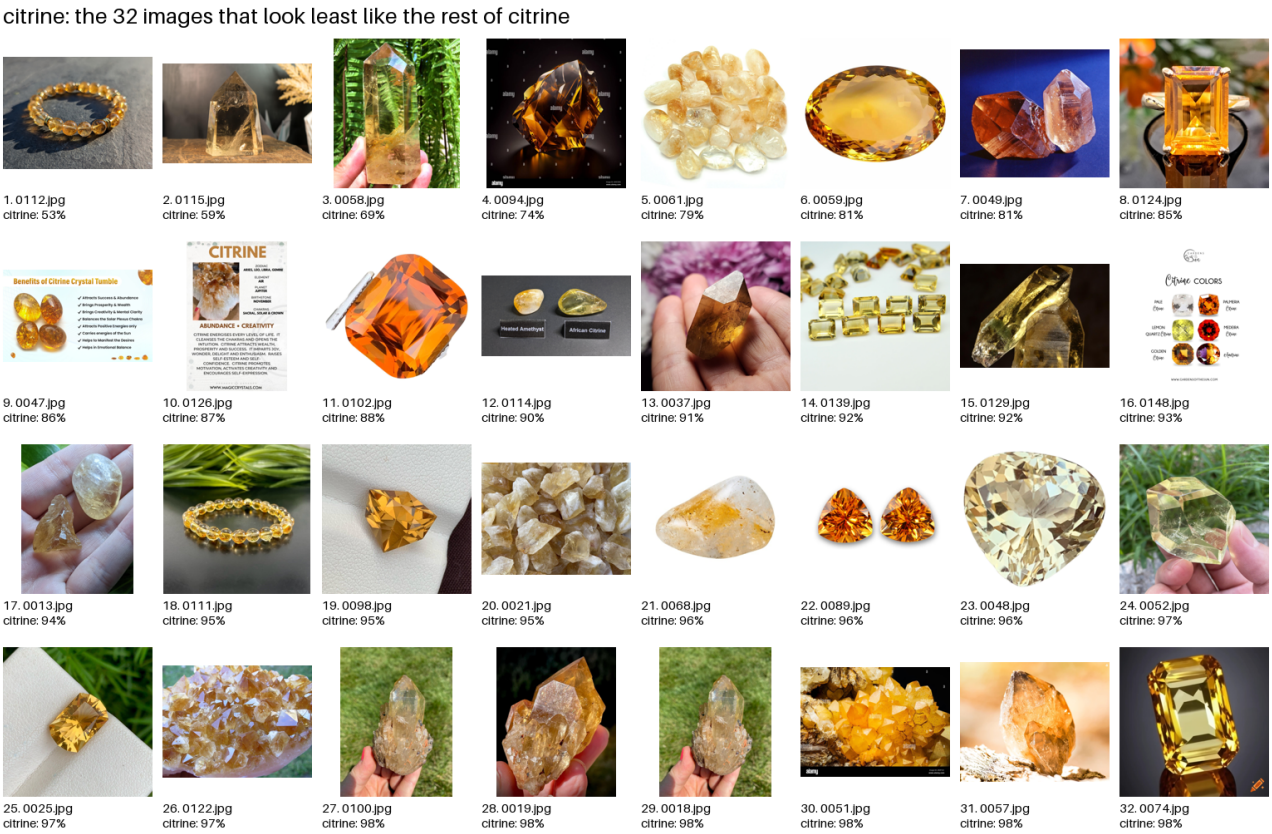
- `citrine/6dc1dd3c`: the crystal is pale, less full in colour and small in size, and the background has colour hues in it, so it could affect the model's answer.
- `citrine/58f05f6d`: the crystal is small, the light is dark and the background has pink colour hues in it.
- `citrine/2788e4fa`: similarly to `58f05f6d`, the crystal is small and the light is dark; however, its colour is fuller.

What is different about my new images

Compared with the downloads, my new images are less saturated, and their colours are sometimes less vivid. The crystals are smaller, usually in the middle of the photo. The light is darker in some of them. Some photos also show my hand as the surface the crystal is lying on.

My guess at why cleaning did not make the model better on my new images: some of the photos I removed because they broke my rules may have looked more like my own photos, and the ones that remained look even more different from my photos. Not all of the removed images were obvious junk. The images I removed are the ones that broke my rules, plus the near copies I chose to remove. Not every suspect was removed: some of the images the tool listed as least like the rest of their category did not break my rules, so I kept them. However, when I check the removed images, they do not necessarily look like my photos. What I can see is that in the clean images the colours are more vivid and the crystals are bigger in size than in my photos.

The citrine suspect sheet is different from the others. Even the downloaded citrines that look least like the rest are still mostly over 90% citrine (21 of 32), and none of them looks like another category. On the other three sheets, many images are under 50%. So the downloaded citrines all look very much alike: vivid in colour and clearly visible (the numbers are in `outputs/suspects.csv`).



The citrine suspect sheet, as `clean.py suspects` drew it.

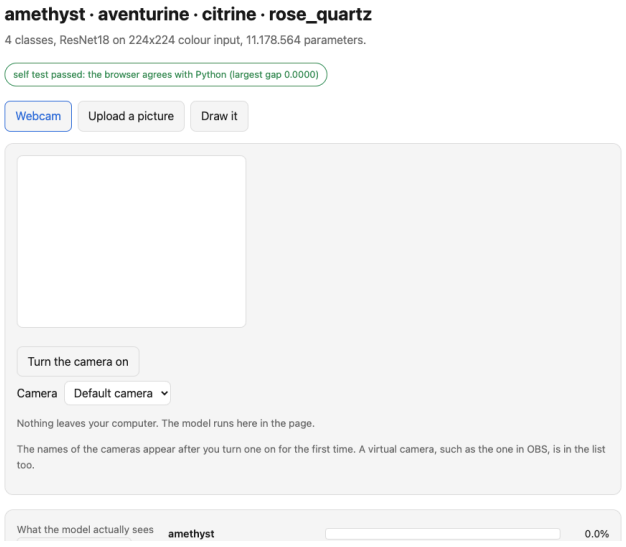
Even so, all three mistakes of the cleaned model on my new images are citrines, while it got all 27 citrines in the downloaded test set right.

My guess: the citrines in my photos are small, less vivid in colour and sometimes paler, rather than having the full yellow or orange colour of the citrines in the clean images, and in two of the three photos it got wrong the light is darker.

Problem 6. The demo

Address: `https://giuliag9.github.io/MAS1004-assignment1/`

I checked the page on my phone: it worked, and it showed the green badge.



My page, with the green badge: “self test passed: the browser agrees with Python”.

I showed the page photos of four things that are none of my categories, and it named one of my four stones every time, because it has no answer for “none of these”: an adapter was rose quartz (47.1%, then amethyst 43.7%), a yellow highlighter was aventurine (65.3%, then citrine only 24.9%), a pink mask product was rose quartz (92.2%), and a beige mug was rose quartz (79.4%).



The yellow highlighter: aventurine 65.3%, citrine 24.9%.

Problem 7. What the model looks at

Guess 1: colour separates amethyst from aventurine

My guess, in my words as I gave it: “I’ll start from my original guess on what the model would look at to distinguish the crystal. I mentioned I expect it to look at colors. From my images, amethyst and aventurine were not mistaken in the current model. So my first guess is that if we lower the saturation on amethyst and aventurine the model will have more problems to distinguish them.”

The change: the saturation of all ten of my new amethyst and aventurine photos lowered to 0%, which leaves the light and dark, the shape and the texture of each photo as they were and takes the colour away completely, so that I can test the model with the absence of colour. Ten photos, five of each class, and all ten originals are answered right by the page.

My requests to the agent, word for word:

> make a copy of each image in data/new_images/amethyst and data/new_images/aventurine with lowered saturation and save the copies, respectively, in data/changed/less_saturation_amethyst and data/changed/less_saturation_aventurine

and, asked by how much:

> to 0% , so we can test with the absence of color.

The agent wrote the change as a short piece of code that I ran on Colab, where my photos are; it made the changed copies and left the originals as they were. I uploaded every original and every changed photo to my web page and wrote down its first two answers:

photo	original	saturation 0%
`amethyst/44db3aaf`	amethyst 96.7%, rose quartz 2.4%	amethyst 84.6%, aventurine 7.9%
`amethyst/a95f9f93`	amethyst 86.9%, rose quartz 8.1%	amethyst 77.0%, aventurine 14.3%
`amethyst/IMG_3072`	amethyst 80.2%, rose quartz 16.7%	**aventurine 45.5%**, amethyst 36.4%
`amethyst/IMG_3089`	amethyst 90.5%, aventurine 5.2%	amethyst 40.8%, aventurine 30.6%
`amethyst/IMG_3200`	amethyst 94.8%, rose quartz 2.3%	**aventurine 45.6%**, amethyst 42.8%
`aventurine/59ad40f0`	aventurine 99.3%, amethyst 0.4%	aventurine 62.9%, amethyst 23.7%
`aventurine/aba4d635`	aventurine 97.3%, amethyst 1.7%	aventurine 82.1%, amethyst 12.5%
`aventurine/IMG_3068`	aventurine 75.2%, rose quartz 15.6%	aventurine 51.5%, rose quartz 33.7%
`aventurine/IMG_3196`	aventurine 80.9%, rose quartz 13.3%	**rose quartz 47.1%**, aventurine 44.6%
`aventurine/IMG_3197`	aventurine 88.7%, rose quartz 9.9%	**rose quartz 62.3%**, aventurine 34.6%

The originals are 10 of 10 right; with the colour taken away, 6 of 10 are still right, and the probability of the right answer is lower for all ten. The four that changed: two amethysts became aventurine, and two aventurines became rose quartz.



amethyst/44db3aaf, original: amethyst 96.7%, rose quartz 2.4%



saturation 0%: amethyst 84.6%, aventurine 7.9%



amethyst/a95f9f93, original: amethyst 86.9%, rose quartz 8.1%



saturation 0%: amethyst 77.0%, aventurine 14.3%



amethyst/IMG_3072, original: amethyst 80.2%, rose quartz 16.7%



saturation 0%: **aventurine 45.5%**, amethyst 36.4%



amethyst/IMG_3089, original: amethyst 90.5%, aventurine 5.2%



saturation 0%: amethyst 40.8%, aventurine 30.6%



amethyst/IMG_3200, original: amethyst 94.8%, rose quartz 2.3%



saturation 0%: **aventurine 45.6%**, amethyst 42.8%



aventurine/59ad40f0, original: aventurine 99.3%, amethyst 0.4%



saturation 0%: aventurine 62.9%, amethyst 23.7%



aventurine/aba4d635, original: aventurine 97.3%, amethyst 1.7%



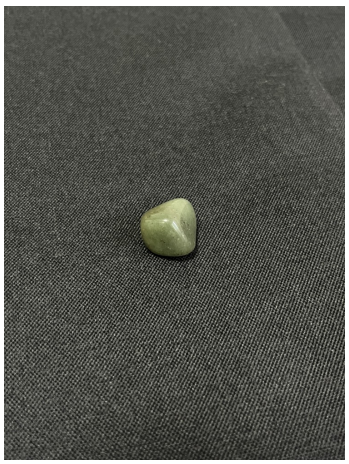
saturation 0%: aventurine 82.1%, amethyst 12.5%



aventurine/IMG_3068, original: aventurine 75.2%, rose quartz 15.6%



saturation 0%: aventurine 51.5%, rose quartz 33.7%



aventurine/IMG_3196, original: aventurine 80.9%, rose quartz 13.3%



saturation 0%: **rose quartz 47.1%**, aventurine 44.6%



aventurine/IMG_3197, original: aventurine 88.7%, rose quartz 9.9%



saturation 0%: **rose quartz 62.3%**, aventurine 34.6%

What the results say about the guess, in my words: From these results I can observe two things. First, the cases in which the 0% saturation changed the model's answer as I thought were only amethyst to aventurine, and not aventurine to amethyst: the only two times the answer changed for aventurine, it went to rose quartz and not to amethyst. Second, by looking at the changed images I realised that the aventurine images that were called rose quartz at 0% saturation were paler than the others, meaning that in the original the colour was lighter, paler, less full. At the same time the amethyst images that were called aventurine at 0% saturation also showed some paler parts and appeared less full, but still darker than the aventurine images that were called rose quartz. Overall the answers change a bit after removing the colour, but not as much as I expected, which makes me think that colour is considered, but it is not the determining element for the model to tell the crystals apart.

Guess 2: transparency, aventurine pale but dark

My guess, in my words as I gave it: "Given what I observed from the first guess, color is not the determinant element for the model. I start to think that transparency also is important when guessing the crystals. As I mentioned in the first guess, 0% saturation aventurine that were guessed to be rose quartz were paler and less full in color, while the amethyst 0% saturation that were guessed to be aventurine also were less full, paler but overall darker. Which makes me think that for the model aventurine is dark, but also pale, while rose quartz is pale but lighter. If I think of the mistakes made, the first one was of data/new_images/citrine/6dc1dd3c-0216-45b8-abb7-214d633d280e.JPG being guessed as rose quartz, and in fact the crystal in the photo is less full in color, and lighter. To check this, the transparency of the images should be lowered. In particular my guess to check if transparency plays a role in how the model distinguishes the crystals, is that if we lower the transparency of the

images of amethyst we turned to 0% saturation, since they're dark, they would be guessed as aventurine (pale but dark)."

The change: a photo has no setting for how see-through a stone is (the "transparency" setting a picture file can have is not seen by the model). So I chose to show less transparency by lowering the contrast to 50%, on the whole picture, on the three amethyst photos that were still called amethyst at 0% saturation. Lowering the contrast brings the light and dark parts of the picture closer to grey, so the crystal looks paler and less full. I chose 50% as a compromise, to make the crystal paler without its dark parts getting too much lighter.

My requests to the agent, word for word:

> So make a copy of data/changed/less_saturation_amethyst/44db3aaf-5cdf-4004-9255-868fefec64c3.JPG, data/changed/less_saturation_amethyst/a95f9f93-5117-4b70-9e92-4c513bb5decc.JPG and data/changed/less_saturation_amethyst/IMG_3089.JPG with lower transparency and save the copies in data/changed/less_saturation_less_transparency_amethyst

and, after the agent explained that a picture has no transparency setting the model would see:

> Since there is no transparency setting to change in the images, I think the best way to make the crystal paler and less full would be to lower contrast to 50%, this way it's enough to make the image paler but keeps the darkess of the crystal which in my guess is what determines aventurine (pale but dark). So make a copy of data/changed/less_saturation_amethyst/44db3aaf-5cdf-4004-9255-868fefec64c3.JPG, data/changed/less_saturation_ame a95f9f93-5117-4b70-9e92-4c513bb5decc.JPG and data/changed/less_saturation_amethyst/IMG_3089.JPG with contrast to 50% and save the copies in data/changed/less_saturation_less_transparenc Also maintain the whole picture of the images for this guess.

The agent wrote the change as a short piece of code that I ran on Colab. My web page's first two answers, next to the ones the same photos got at 0% saturation in Guess 1:

photo	saturation 0%	saturation 0% and contrast 50%
`amethyst/44db3aaf`	amethyst 84.6%, aventurine 7.9%	**rose quartz 50.0%**, amethyst 27.7%
`amethyst/a95f9f93`	amethyst 77.0%, aventurine 14.3%	**rose quartz 46.2%**, aventurine 27.8%
`amethyst/IMG_3089`	amethyst 40.8%, aventurine 30.6%	**rose quartz 64.4%**, aventurine 25.2%

All three changed their answer, all three to rose quartz and none to aventurine; aventurine is second for two of them.



44db3aaf, saturation 0%: amethyst 84.6%, aventurine 7.9%



and contrast 50%: **rose quartz 50.0%**, amethyst 27.7%



a95f9f93, saturation 0%: amethyst 77.0%, aventurine 14.3%



and contrast 50%: **rose quartz 46.2%**, aventurine 27.8%



IMG_3089, saturation 0%: amethyst 40.8%, aventurine 30.6%



and contrast 50%: **rose quartz 64.4%**, aventurine 25.2%

What the results say about the guess, in my words: These results make me think that while lowering the contrast, the crystals also got lighter, and therefore closer to rose quartz rather than aventurine. Lowering the contrast and keeping the original darkness is difficult. However, these results seem to fit my idea that rose quartz is pale and light. Even if it did not separate amethyst from aventurine, transparency, which in this case was conveyed by lowering the contrast, seems to be considered by the model: all three images got a different answer from the one they had at 0% saturation. It is also worth noticing that the lower contrast changed all three answers, but it was applied to the 0% saturation images, not to the original ones. Overall, the results of the second guess make me think of two things. First, if rose quartz is light and pale, and the mistaken citrine is also light and pale, what does the model look at in the image to tell them apart, and why was the citrine called rose quartz? Perhaps transparency is not the answer. Second, there might be another factor that the model considers to tell the crystals apart, or a combination of factors.

Guess 3: more colour would help the pale citrines

My guess, in my words as I gave it: "Given the previous guesses results, I think color and transparency are in a way considered by the model, even if not in the way I expected them. At this point is good to consider the case of the 3 mistakes of citrine of my images. Starting from the one I mentioned, citrine 6dc1dd3c-0216-45b8-abb7-214d633d280e.JPG was guessed as rose quartz, and these crystals are generally light and pale. As I previously said, the images of citrine in clean, are more vivid in color than my own photos, which I think is an important element that makes the images in clean different from mine. In the first guess I focused on color, and observed if its absence (so a 0% saturation) could impact the model's guessing, and overall, there were some changes, even for the images that were guessed correctly (guessed with less percentage). This makes me still want to keep color in

consideration for a guess. Therefore, it would be interesting to test the opposite of what I did in the first guess, meaning instead of reducing color, enhancing it. Since both citrine and rose quartz are pale, the aspect that is different in them is color and them being mistaken could be linked to the color not being vivid. So, my third guess is that if citrine has higher saturation the model would have less problem recognizing it. If this guess turned to be true, it would confirm the reason for the mistake in my images."

What I expected, written before I saw the results: "If my guess is right the 3 wrong will turn right and the other 2 right ones will stay right with higher percentages."

The change: 70% more colour (saturation 170% on the scale of Guess 1, where 100% is the photo as it is), on the whole picture, on all five of my new citrine photos: the three the model gets wrong and the two it gets right.

My requests to the agent, word for word:

> So make a copy of each image in data/new_images/citrine with higher saturation (70%) and save the copies in data/changed/saturated_citrine

and, asked whether 70% meant 70% more colour:

> yes 70% more color.

The agent wrote the change as a short piece of code that I ran on Colab. Raising the saturation makes the colour that is already there stronger; it cannot add colour to parts that are white or grey. My web page's first two answers:

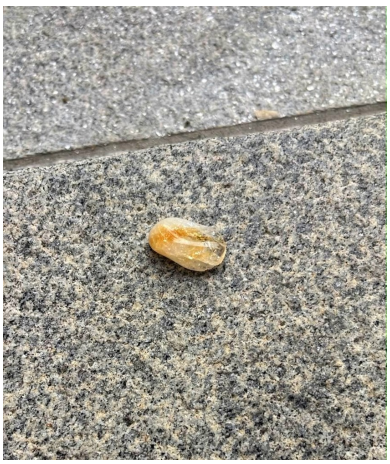
photo	original	saturation +70%
`citrine/6dc1dd3c`	rose quartz 52.8%, aventurine 26.5%	rose quartz 37.7%, aventurine 31.5%
`citrine/58f05f6d`	rose quartz 49.1%, aventurine 22.4%	rose quartz 54.7%, aventurine 18.0%
`citrine/2788e4fa`	rose quartz 31.0%, amethyst 29.8%	rose quartz 33.0%, amethyst 25.1%
`citrine/08f618d6`	citrine 89.5%, aventurine 4.7%	citrine 96.2%, aventurine 2.1%
`citrine/f16efa23`	citrine 59.9%, amethyst 25.1%	citrine 94.1%, amethyst 4.1%

The two that were right stayed right and became more sure, `f16efa23` from 59.9% to 94.1%. The three that were wrong stayed wrong, all three still rose quartz, and citrine is not among the first two answers for any of them.

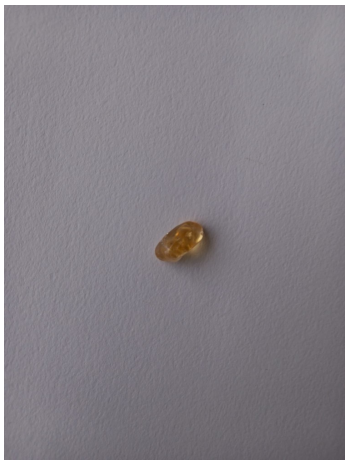
The page's numbers for the three originals are close to, but not exactly the same as, the ones `check.py` printed in Problem 5 (54%, 51% and 32%). For `2788e4fa` the order of the first two even changes: `check.py` said amethyst, the page says rose quartz 31.0% with amethyst 29.8% second. Before the model looks at a photo, the photo is shrunk to a small square; the web page and Python shrink it in slightly different ways, so the model gets an almost, but not exactly, identical picture. The green badge cannot notice this, because its test pictures are already small squares. When the model is unsure, as it is here, such a tiny difference is enough to swap its first two answers.



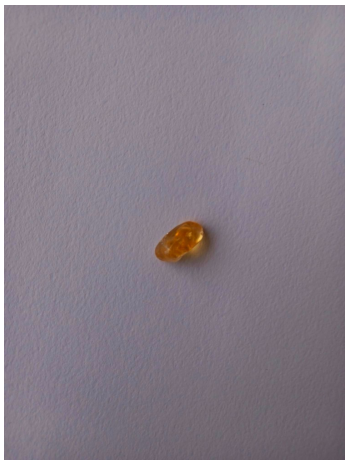
citrine/6dc1dd3c, original: rose quartz 52.8%, aventurine 26.5%



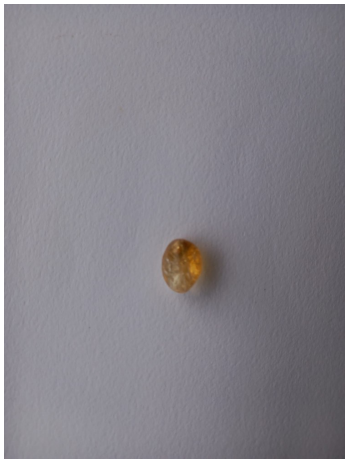
saturation +70%: rose quartz 37.7%, aventurine 31.5%



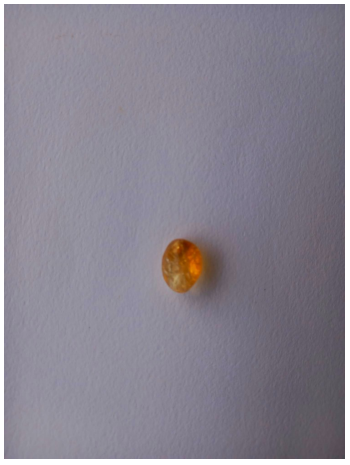
citrine/58f05f6d, original: rose quartz 49.1%, aventurine 22.4%



saturation +70%: rose quartz 54.7%, aventurine 18.0%



citrine/2788e4fa, original: rose quartz 31.0%, amethyst 29.8%



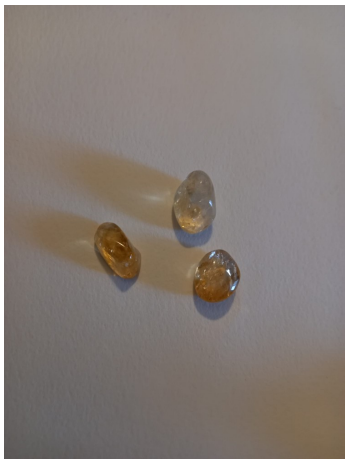
saturation +70%: rose quartz 33.0%, amethyst 25.1%



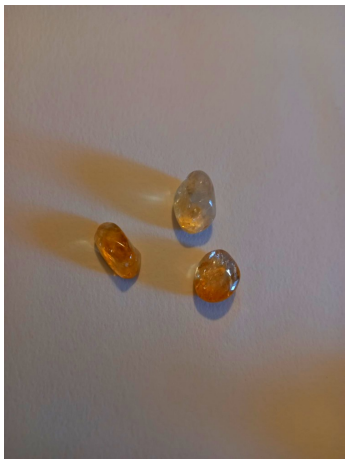
citrine/08f618d6, original: citrine 89.5%, aventurine 4.7%



saturation +70%: citrine 96.2%, aventurine 2.1%



citrine/f16efa23, original: citrine 59.9%, amethyst 25.1%



saturation +70%: citrine 94.1%, amethyst 4.1%

What the results say about the guess, in my words: Once again, the results do not confirm my guess completely; in fact they only confirmed my guess for the images of citrine that were already guessed right. However, the results give me more insight about where I should put my attention in understanding what the model looks at to distinguish the crystals, in particular what in those 3 images of citrine is different from the others. As the saturation was increased, other colours in the image also became more evident. In particular, it is possible to notice that the background's colour is also more vivid. In the saturated image 6dc1dd3c-0216-45b8-abb7-214d633d280e.JPG the granite background shows more hues that can resemble pink, but also a slight green. In the saturated image 58f05f6d-77f2-467e-b497-c4265bb43b8e.JPG, due to the type of light, the background looks more pink too. The same happens in the saturated image 2788e4fa-a7ad-453a-b3ad-3984b54d6e2c.JPG. In fact, the results show that for the saturated images 58f05f6d-77f2-467e-b497-c4265bb43b8e.JPG and 2788e4fa-a7ad-453a-b3ad-3984b54d6e2c.JPG the model is surer about its answer "rose quartz", while for 6dc1dd3c-0216-45b8-abb7-214d633d280e.JPG the first answer, rose quartz, decreases to 37.7% as aventurine rises to 31.5%. Therefore, the conclusion I draw from the results of the third guess is that the colour of the background, also influenced by light, can influence the model's answer in some situations.

Guess 4: the stone's size and the background

My guess, in my words as I gave it: "From the third guess I concluded that the color of background can be a factor that influences the model. Given all the previous guesses, I start to consider more and more the idea that the model uses a combination of elements to guess the crystals. The images that were still guessed wrong also had another thing in common: the crystals are small. Additionally, the two citrine images that are guessed right are the only two with more than one crystal in them.

To check both background and size at the same time, the images could be cut so that the crystal occupies the most part of the image, and has little background. My guess would be that if we cut the background from the images of citrine, making the crystal the main object in the image, the model could guess them correctly."

The change: the three citrines that were still wrong in Guess 3, in their saturated versions, cut to a square around the stone, so that the stone fills most of the picture with only a thin border of background (the square is 1.5 times as wide as the stone). A square, because the model always looks at a square, so it sees all of the cut picture.

My request to the agent, word for word:

> So make a copy of images data/changed/saturated_citrine/6dc1dd3c-0216-45b8-abb7-214d633d280e.JPG, data/changed/saturated_citrine/2788e4fa-a7ad-453a-b3ad-3984b54d6e2c.JPG and data/changed/saturated_citrine/58f05f6d-77f2-467e-b497-c4265bb43b8e.JPG cut the background around the crystals and save the copies in data/changed/saturated_cut_citrine. In this case still work on the saturated version because I still think that saturation of the crystal plays a role.

The agent found the stone in each picture, chose the square, showed me the three cut pictures before they were made, and wrote the cut as a short piece of code that I ran on Colab. My web page's first two answers, next to the ones the same saturated pictures got in Guess 3:

photo	saturated	saturated and cut
`citrine/6dc1dd3c`	rose quartz 37.7%, aventurine 31.5%	**citrine 75.3%**, aventurine 12.0%
`citrine/58f05f6d`	rose quartz 54.7%, aventurine 18.0%	**citrine 99.6%**, amethyst 0.3%
`citrine/2788e4fa`	rose quartz 33.0%, amethyst 25.1%	**citrine 97.9%**, amethyst 0.9%

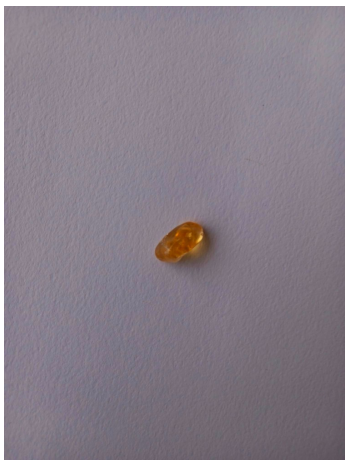
All three changed from rose quartz to citrine, two of them above 97%. The pictures were both saturated and cut, so these three results alone cannot separate the two changes; but in Guess 3 the same three pictures, saturated and not cut, were all still rose quartz.



6dc1dd3c, saturated: rose quartz 37.7%, aventurine 31.5%



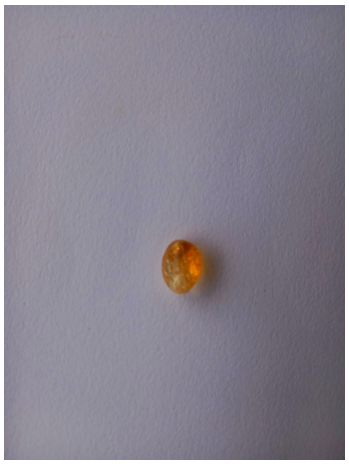
saturated and cut: citrine 75.3%, aventurine 12.0%



58f05f6d, saturated: rose quartz 54.7%, aventurine 18.0%



saturated and cut: citrine 99.6%, amethyst 0.3%



2788e4fa, saturated: rose quartz 33.0%, amethyst 25.1%



saturated and cut: citrine 97.9%, amethyst 0.9%

What the results say about the guess, in my words: These results are the first in which my guess was completely right. I can therefore assume that cutting the background and making the crystal the main object of the image is an important element that the model considers in order to answer correctly. The answers on the cut images are so accurate that it makes me wonder whether it plays a bigger role than colour (considering it was done on saturated images).

Guess 5: the cut on its own, on all my photos

My guess, in my words as I gave it: "At this point, I think a final check should be done, with a 5th guess. Guess 4 showed that making the crystal bigger, and cutting the background was important for the model to guess, finally, the 3 citrine correctly. However, in those images the color also was saturated from the previous guess. So it could be interesting to test the cutting generally on the images. In particular, applying the cut to all of my original images. Since the other crystals were guessed right, observing how only the cut affects the results can be useful to understand how much cutting the background and making the crystal the main object in the image is taken in consideration from the model. Also, because the images are original, so with normal saturation, observing the results of the 3 citrine, can give an idea of whether cut plays a bigger role than color. So my 5th guess is that if we cut the background from my original images, making the crystals the main object in the image, the 3 citrine 6dc1dd3c-0216-45b8-abb7-214d633d280e.JPG, 58f05f6d-77f2-467e-b497-c4265bb43b8e.JPG, and 2788e4fa-a7ad-453a-b3ad-3984b54d6e2c.JPG would still be guessed correctly, and all the other images (that were previously guessed right) would also be guessed right and more accurately."

The change: all 21 of my original photos, with their normal colour, cut in the same way as in Guess 4: a square around the stone (or around all the stones, when there are several), so that the stone fills

most of the picture. The agent showed me all 21 cuts before they were made, and on my request moved the square for `amethyst/a95f9f93` lower, so that the stone is not cut.

My request to the agent, word for word:

> So make a copy of each image in data/new_images cut the background around the crystals and save the copies in data/changed/cut

and, after the preview:

> yes, just change the cut on image a95f9f93-5117-4b70-9e92-4c513bb5decc.JPG to make the cut a bit lower so that the crystal is not cut.

The copies are in `data/changed/cut/<class>/`, one folder per class. The agent wrote the cut as a short piece of code that I ran on Colab. My web page's first two answers, the originals as the page answered them (amethyst and aventurine from Guess 1, citrine from Guess 3, rose quartz read for this guess):

photo	original	cut
`amethyst/44db3aaf`	amethyst 96.7%, rose quartz 2.4%	amethyst 87.0%, rose quartz 10.5%
`amethyst/a95f9f93`	amethyst 86.9%, rose quartz 8.1%	amethyst 93.3%, rose quartz 3.7%
`amethyst/IMG_3072`	amethyst 80.2%, rose quartz 16.7%	amethyst 62.1%, rose quartz 29.2%
`amethyst/IMG_3089`	amethyst 90.5%, aventurine 5.2%	amethyst 88.7%, aventurine 8.9%
`amethyst/IMG_3200`	amethyst 94.8%, rose quartz 2.3%	amethyst 91.1%, rose quartz 5.7%
`aventurine/59ad40f0`	aventurine 99.3%, amethyst 0.4%	aventurine 99.9%, rose quartz 0.1%
`aventurine/aba4d635`	aventurine 97.3%, amethyst 1.7%	aventurine 98.8%, amethyst 0.7%
`aventurine/IMG_3068`	aventurine 75.2%, rose quartz 15.6%	aventurine 95.9%, rose quartz 3.3%
`aventurine/IMG_3196`	aventurine 80.9%, rose quartz 13.3%	aventurine 98.8%, rose quartz 0.7%
`aventurine/IMG_3197`	aventurine 88.7%, rose quartz 9.9%	aventurine 98.5%, rose quartz 1.4%
`citrine/6dc1dd3c`	rose quartz 52.8%, aventurine 26.5%	citrine 48.0%, rose quartz 23.5%
`citrine/08f618d6`	citrine 89.5%, aventurine 4.7%	citrine 94.5%, aventurine 2.4%
`citrine/58f05f6d`	rose quartz 49.1%, aventurine 22.4%	citrine 97.2%, amethyst 2.2%
`citrine/2788e4fa`	rose quartz 31.0%, amethyst 29.8%	citrine 90.1%, amethyst 6.4%
`citrine/f16efa23`	citrine 59.9%, amethyst 25.1%	citrine 85.9%, aventurine 6.8%
`rose_quartz/9dacd40a`	rose quartz 99.8%, aventurine 0.1%	rose quartz 99.9%, aventurine 0.1%
`rose_quartz/2270e0ff`	rose quartz 95.3%, aventurine 2.9%	rose quartz 97.6%, aventurine 1.8%
`rose_quartz/ea7f4b76`	rose quartz 99.7%, amethyst 0.2%	rose quartz 99.8%, aventurine 0.1%
`rose_quartz/IMG_3069`	rose quartz 47.0%, amethyst 34.8%	rose quartz 53.3%, aventurine 31.9%
`rose_quartz/IMG_3194`	rose quartz 91.4%, aventurine 6.9%	rose quartz 99.1%, aventurine 0.8%
`rose_quartz/IMG_3204`	rose quartz 76.7%, aventurine 22.2%	rose quartz 63.2%, aventurine 35.7%

The originals are 18 of 21 right on the page; the cut pictures are 21 of 21 right. The three citrines that were wrong are right without any extra colour, `6dc1dd3c` at 48.0% against 75.3% when it was also saturated in Guess 4. Of the 18 photos that were right before, 13 are more sure after the cut and 5 are less sure: four amethysts (`44db3aaf`, `IMG_3072`, `IMG_3089`, `IMG_3200`) and one rose quartz (`IMG_3204`).



amethyst/44db3aaf, original: amethyst 96.7%, rose quartz 2.4%



cut: amethyst 87.0%, rose quartz 10.5%



amethyst/a95f9f93, original: amethyst 86.9%, rose quartz 8.1%



cut: amethyst 93.3%, rose quartz 3.7%



amethyst/IMG_3072, original: amethyst 80.2%, rose quartz 16.7%



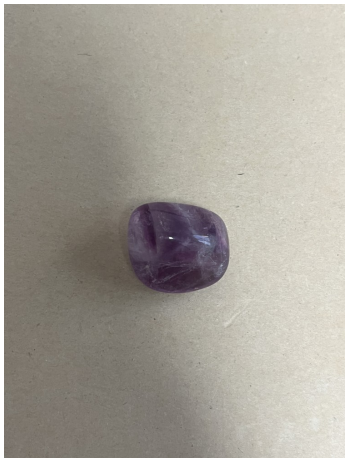
cut: amethyst 62.1%, rose quartz 29.2%



amethyst/IMG_3089, original: amethyst 90.5%, aventurine 5.2%



cut: amethyst 88.7%, aventurine 8.9%



amethyst/IMG_3200, original: amethyst 94.8%, rose quartz 2.3%



cut: amethyst 91.1%, rose quartz 5.7%



aventurine/59ad40f0, original: aventurine 99.3%, amethyst 0.4%



cut: aventurine 99.9%, rose quartz 0.1%



aventurine/aba4d635, original: aventurine 97.3%, amethyst 1.7%



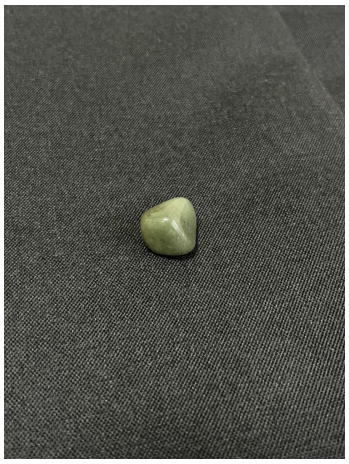
cut: aventurine 98.8%, amethyst 0.7%



aventurine/IMG_3068, original: aventurine 75.2%, rose quartz 15.6%



cut: aventurine 95.9%, rose quartz 3.3%



aventurine/IMG_3196, original: aventurine 80.9%, rose quartz 13.3%



cut: aventurine 98.8%, rose quartz 0.7%



aventurine/IMG_3197, original: aventurine 88.7%, rose quartz 9.9%



cut: aventurine 98.5%, rose quartz 1.4%



citrine/6dc1dd3c, original: rose quartz 52.8%, aventurine 26.5%



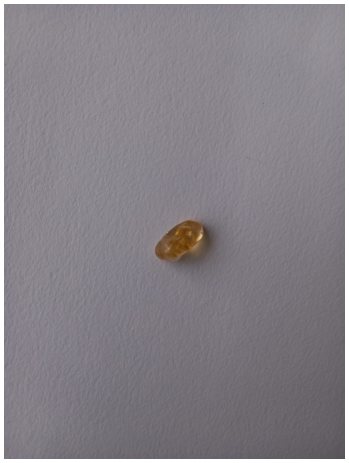
cut: citrine 48.0%, rose quartz 23.5%



citrine/08f618d6, original: citrine 89.5%, aventurine 4.7%



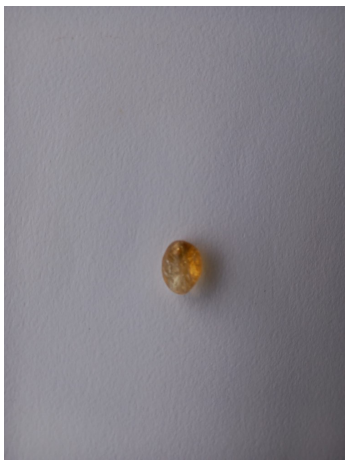
cut: citrine 94.5%, aventurine 2.4%



citrine/58f05f6d, original: rose quartz 49.1%, aventurine 22.4%



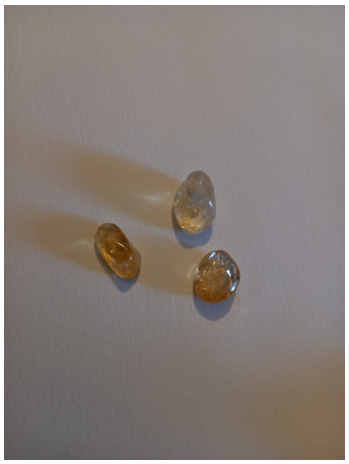
cut: citrine 97.2%, amethyst 2.2%



citrine/2788e4fa, original: rose quartz 31.0%, amethyst 29.8%



cut: citrine 90.1%, amethyst 6.4%



citrine/f16efa23, original: citrine 59.9%, amethyst 25.1%



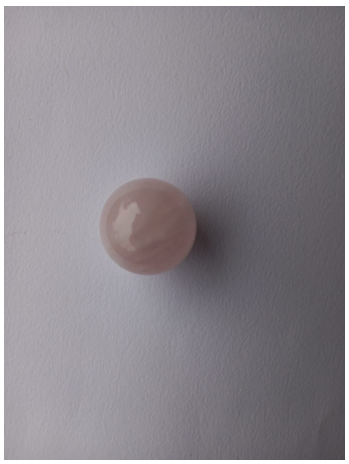
cut: citrine 85.9%, aventurine 6.8%



rose quartz/9dacd40a, original: rose quartz 99.8%, aventurine 0.1%



cut: rose quartz 99.9%, aventurine 0.1%



rose quartz/2270e0ff, original: rose quartz 95.3%, aventurine 2.9%



cut: rose quartz 97.6%, aventurine 1.8%



rose quartz/ea7f4b76, original: rose quartz 99.7%, amethyst 0.2%



cut: rose quartz 99.8%, aventurine 0.1%



rose quartz/IMG_3069, original: rose quartz 47.0%, amethyst 34.8%



cut: rose quartz 53.3%, aventurine 31.9%



rose quartz/IMG_3194, original: rose quartz 91.4%, aventurine 6.9%



cut: rose quartz 99.1%, aventurine 0.8%



rose quartz/IMG_3204, original: rose quartz 76.7%, aventurine 22.2%



cut: rose quartz 63.2%, aventurine 35.7%

What the results say about the guess, in my words: From these results I can observe two things. First, as I expected, the 3 wrong citrines were guessed correctly even without being more saturated, although with less accuracy. This means that cutting the background and making the crystal bigger in the image is important for the model to guess it correctly; in particular it plays a bigger role than colour, since in Guess 3 the 3 citrines were still guessed wrong when only the saturation was enhanced. Second, the crystals that were already guessed right were also guessed right, as expected, but not exactly as I imagined: not all the answers were more accurate. In particular, 4 amethysts were guessed with less accuracy and only one with more. This is important to give the right amount of importance to the cut (so to the size of the crystals and the absence of background). Considering the other factors that I observed in the previous guesses, such as transparency and colour, and my idea of a combination of elements, the reason why the 4 amethysts got a less accurate answer after the cut could be connected to the fact that the colour of amethyst is fuller and darker, which can have an effect on how useful the cut is to the model in that case. However, it is important to also mention that one of the rose quartz photos, IMG_3204.jpg, also had less accuracy after the cut, but in this case the transparency and saturation of the crystal are different from amethyst. Therefore, it is possible that there is another element that the model looks at that is not among those I tried to test in my 5 guesses.

What I now think the model uses

At the beginning of the assignment I thought that the model would mostly look at the colour of the crystals to distinguish them, as it is the first thing that is visible by looking at them. However, through the 5 guesses I made I became more sure that the model does not use only one element to recognise the different crystals, but rather takes a combination of them into consideration. This is probably the part of the conclusion that I am most sure about.

The elements that I tried to test, in order to understand whether they are considered by the model, were three. The first was colour, so the saturation of the crystals. The second was transparency, in the sense of how pale the crystal is, and it was observed through less contrast. Lastly, the size of the crystal and the reduction of the background, which were observed together through the cut. Among these, I think that mainly two elements are considered by the model, but differently. The element that I think has more importance for the model is the size of the crystal and the reduction of the background, since all my images were recognised correctly after the cut that made the crystal the main object in the image and reduced the background. Colour (saturation) is the second element in importance, considering that the 3 saturated citrines were not recognised correctly before the cut. As for transparency, it might play a role, but it was not easy to check, as lowering the contrast is not exactly the same as having a crystal that is pale and less full in colour. Therefore, from my guesses I conclude that the model uses a combination of the size of the crystal, the reduction of the background, and colour to distinguish the categories, and that the size of the crystal and the reduction of the background have a stronger impact on the model than colour.

However, it is important to mention that my guesses were based on tests I tried on my own images, which were only 21, so my conclusion has limitations. As mentioned in the conclusion of Guess 5, I think there might be other elements that the model looks at to distinguish the categories, which I did not test in my guesses. Apart from Guess 5, which tested the cut on all my images, my other guesses were done on a smaller number of images; in particular, my second and fourth guesses were more specific and focused only on 3 amethysts and on the 3 wrong citrines, so my conclusions from those two guesses are limited. I am aware that more tests and guesses, especially on more images and from other sources as well, could give a more complete answer to what the model uses to recognise the categories.

Appendix. Output of check.py

(The warning that `docs/model.onnx` is not committed was true when I ran `check.py` on Colab; I committed and pushed the model from my own computer afterwards, and the published page uses it, see Problem 6.)

```
MAS1004 Assignment 1, checking your work

Training images  (/content/drive/MyDrive/mas1004/assignment1/data/clean)
------------------------------------------------------------------------
  amethyst                        94 images
  aventurine                      55 images
  citrine                        133 images
  rose_quartz                     92 images

Your images from a new source  (/content/drive/MyDrive/mas1004/assignment1/data/new_images)
------------------------------------------------------------------------------------------
  amethyst                         5 images
  aventurine                       5 images
  citrine                          5 images
  rose_quartz                      6 images

The same image appearing twice
------------------------------
  ok       no file appears twice byte for byte. Near copies (the same picture resized or re-saved)
  ↪  are only found by `python src/clean.py suspects` and `python src/clean.py overlap`.

The model you exported for the web page
---------------------------------------
  ok       4 classes: amethyst, aventurine, citrine, rose_quartz
  ok       model.onnx is 44.8 MB, 11,178,564 parameters
           input [1, 3, 224, 224], prepared as: shorter side 256, centre 224 x 224, mean [0.485,
           ↪  0.456, 0.406], std [0.229, 0.224, 0.225]
  warning  docs/model.onnx is not committed. Your published page will not have a model until you
  ↪  commit and push it.
```

```
Does the exported model give the answers Python gave?
------------------------------------------------------
  ok       yes, they agree (largest gap 0.0000)

Your images from a new source  (/content/drive/MyDrive/mas1004/assignment1/data/new_images)
-----------------------------------------------------------------------------------------

  accuracy on your new images: 18/21 = 85.7%

  rows are what it really is, columns are what the model said
                 amethyst aventuri  citrine rose_qua
    amethyst            5        0        0        0
    aventurine          0        5        0        0
    citrine             1        0        2        2
    rose_quartz         0        0        0        6

  the mistakes it was most sure about:
    6dc1dd3c-0216-45b8-abb7-214d633d280e.JPG said rose_quartz (54%), really citrine
    58f05f6d-77f2-467e-b497-c4265bb43b8e.JPG said rose_quartz (51%), really citrine
    2788e4fa-a7ad-453a-b3ad-3984b54d6e2c.JPG said amethyst (32%), really citrine

Summary
-------
  nothing is broken
  1 thing(s) worth a sentence in your report:
    - docs/model.onnx is not committed. Your published page will not have a model until you commit
    ↪  and push it.
```